Week 2 Implementation Plan

OpenClaw + Stratus: Counterfactual Guardrail for Healthcare Access Scheduling

Track

Stratus X1 Hackathon
Track: `Multi-Agent Systems`

0. Objective

Week 2 should deliver a **90% final product**, not another prototype.

By the end of this week, the demo should prove three things:

1. **Real incident**
   - a high-demand telehealth or specialist-slot release creates a live-looking access incident
   - the issue is observed through dashboards, alerts, browser-visible state, and business metrics
2. **Counterfactual guardrail**
   - the dangerous local reflex is explicitly shown and rejected
   - Stratus chooses a safer action after reasoning over a shortlist
3. **True multi-agent workflow**
   - observation is parallelized across four specialist agents
   - OpenClaw executes the chosen action
   - post-action evaluation writes a case back into the case library

Product claim:

> We are building a Counterfactual Remediation Guardrail for healthcare access operations: a system that predicts whether a remediation is safe before acting, then verifies whether the prediction held after action.

This should feel like a **world-model safety layer for operational agents**, not a generic incident copilot.

0.1 Final Product Snapshot

By demo day, the product should look like one coherent healthcare-access operations system with five visible pieces:

1. **Telehealth Scheduling Stability Board**
   - a business-facing operations page at `/operations`
   - shows portal demand, eligibility health, slot pressure, abandonment, and booking completion
   - makes the incident feel like a real patient-access problem, not a toy metrics page
2. **Guardrail Console**
   - the operator-facing control shell at `/`
   - contains Incident, Decision, Execution, and Verdict views
   - shows the shortlist, Stratus ranking, browser handoff, and final verdict
3. **OpenClaw Execution Surface**
   - the dedicated browser page at `/openclaw-execution`
   - gives OpenClaw one stable place to inspect, click, and verify
4. **Armed Watch Layer**
   - OpenClaw is armed once and stays in the background
   - a watcher latches a `pending_incident` from the Alertmanager webhook path
   - OpenClaw wakes up only when a real incident arrives
5. **Case-Aware Decision Loop**
   - parallel sentinel agents observe the situation
   - a shortlist is built from the broader remediation library
   - Stratus ranks only the shortlist and rejects the dangerous reflex
6. **Final Verdict Artifact**
   - one judge-facing report that states:
     - incident summary
     - dangerous reflex rejected
     - safer action chosen
     - predicted vs actual
     - blast radius
     - case writeback status

If the final product is working correctly, the audience should feel that they are looking at:

- a real telehealth scheduling operations board
- a real operator decision console
- a real background monitoring and wake-up path
- a real OpenClaw execution step
- a real closed loop from prediction to verification

1. Week 1 Base We Already Have

We are not starting from zero. The current base already includes:

- an end-to-end OpenClaw + Stratus flow on `workflow`
- live Stratus ranking at the guarded decision step
- a dedicated OpenClaw browser execution surface
- Prometheus-backed verification
- plan, browser playbook, demo artifact, and final report outputs
- a broader remediation library with scenario-specific shortlist selection
- a baseline direction and simulator/comparison shape
- a Guardrail Console shell with Incident, Decision, Execution, and Verdict views

Week 2 is about turning that base into a **healthcare-access product story** with stronger realism, clearer multi-agent boundaries, and a more impressive demo.

2. Why Healthcare Access Scheduling

This is the lowest-churn way to make the product feel more serious without changing the core mechanics.

We keep the stack:

- OpenClaw for orchestration and browser execution
- an Alertmanager/webhook watcher for background activation
- Stratus for counterfactual ranking
- Prometheus for verification
- the current shared state, alert flow, browser playbook, and verdict surfaces

We change the story:

- `frontend` becomes the **patient portal**
- `checkout` becomes the **scheduling service**
- `payment` becomes **eligibility / benefits verification**
- "seat holds" become **slot holds**
- "conversion" becomes **booking completion**
- "queue abandonment" becomes **scheduling abandonment**

This keeps the demo operational, not clinical. We are not claiming autonomous diagnosis or treatment. We are showing how operational agents can protect **patient access and fairness** during digital access incidents.

3. Primary Scenario A

High-Demand Telehealth Slot Release: Scheduling Retry Spiral

It is 9:00 AM and a hospital system opens a limited batch of same-week telehealth and specialist appointments.

Traffic spikes immediately. Patients enter the portal, browse limited slots, and attempt to book. The eligibility / benefits verification dependency becomes slow and flaky, but it is not fully down. The scheduling service is built to be resilient, so it retries. Patients also retry manually. Slot holds remain open while eligibility is unresolved.

What starts as a small dependency degradation becomes a self-amplifying access incident:

- scheduling latency spikes
- retry pressure increases
- slot holds clog limited availability
- abandonment rises
- booking completion drops
- access fairness degrades because a small number of patients occupy scarce slots while many others cannot complete booking

The dangerous local reflex is obvious:

- restart eligibility verification

That reflex is locally understandable, but globally risky during an active retry spiral. Restarting the dependency too early can:

- reintroduce cold capacity into a retry backlog
- extend the backlog instead of shrinking it
- keep slot holds stuck
- worsen access fairness

The product should prove that it can:

- detect this access incident
- reject the dangerous reflex
- choose a safer first remediation
- execute it through OpenClaw
- verify the result and record the case

4. Realism Strategy

We should keep the current engineering base and make the story more realistic by layering healthcare-access semantics on top.

Primary demo base:

- OTel Demo on Kubernetes

Operator-facing semantic layer:

- patient portal
- scheduling service
- eligibility verification dependency
- slot holds
- booking completion
- scheduling abandonment

Optional mirror target:

- if we do not directly integrate a real telehealth stack, we mirror the concepts used by healthcare scheduling systems and FHIR scheduling resources rather than inventing a fake workflow from scratch

That is enough to make the product credible without introducing a new integration risk this late.

5. Incident Portfolio

We should prepare several realistic incident snapshots so the same workflow can choose different actions under different conditions.

Scenario A: Scheduling Retry Spiral

- core problem:

    - eligibility verification degrades and retries amplify across scheduling
- expected symptoms:
    - scheduling latency high
    - retry rate high
    - booking completion down
    - scheduling abandonment up
- likely safer actions:
    - `rate_limit_retries`
    - `enable_payment_circuit_breaker`
    - `increase_retry_backoff`
- dangerous reflex:
    - `restart_payment`

Scenario B: Eligibility Verification Flap

- core problem:
    - eligibility verification is unstable but retry amplification is moderate
- expected symptoms:
    - elevated error rate
    - booking completion down
    - abandonment rising
- likely safer actions:
    - `enable_payment_circuit_breaker`
    - `increase_retry_backoff`
    - `restart_payment` only if backlog pressure is already low

Scenario C: Slot Hold Clog

- core problem:
    - slot holds remain open too long and access is clogged even when core infra is still mostly healthy
- expected symptoms:
    - slot hold utilization high
    - hold expiration high
    - booking completion poor
    - fairness / access skew elevated
- likely safer actions:
    - `disable_flag` shown as `Disable Online Scheduling`
    - `rate_limit_retries`
    - `pause_noncritical_background_jobs`

Scenario D: Regional Access Saturation

- core problem:
    - one region or cluster is close to saturation and naive full traffic shift could spread the incident
- expected symptoms:
    - regional latency skew
    - abandonment elevated
    - moderate retry pressure
- likely safer actions:
    - controlled `shift_traffic`

    - `increase_retry_backoff`
    - localized degradation before full failover

6. Realistic Metrics

The incident portfolio should use metrics that feel real for healthcare access operations.

Core technical metrics:

- scheduling latency p95
- scheduling error rate
- booking retry rate
- dependency timeout rate
- regional saturation / load

Business and access metrics:

- booking completion rate
- scheduling abandonment rate
- slot hold utilization
- slot hold expiration rate
- access skew / fairness pressure

Guardrail health metrics:

- predicted blast radius
- actual blast radius
- decision fidelity
- predicted-vs-actual drift

The simulator should preserve believable relationships:

- higher retry rate should usually worsen latency and abandonment
- higher slot hold utilization should usually worsen access fairness and completion
- stronger throttling or circuit breaking should reduce retry pressure first, then improve latency
- disabling online scheduling should sharply reduce digital load but force manual fallback

7. Multi-Agent Workflow

The main workflow should be:

1. **Parallel observation**
2. **Incident latch / wake-up**
3. **Case-library retrieval**
4. **Shortlist generation**
5. **Stratus guardrail ranking**
6. **OpenClaw execution**
7. **Post-action evaluation**
8. **Case writeback**

7.0 Armed Watch Layer

Before the rest of the workflow runs, OpenClaw should be armed once and remain idle in the background.

The watcher should:

- observe `alerts/latest.json` or the Alertmanager alert state
- latch a `pending_incident` when a new firing alert arrives
- mark the incident `acknowledged` when OpenClaw starts handling it
- mark it `completed` after verify or fallback
- return to watch mode automatically

This is better than a manual trigger prompt because the demo now feels like an operational agent waking up to a real incident instead of being told exactly when to act.

7.1 Observation Layer

Run four specialist agents in parallel.

Eligibility Sentinel Agent

- reads eligibility latency, timeout rate, and dependency health
- determines whether the dependency is degraded, flapping, or truly down

Access & Fairness Sentinel Agent

- reads scheduling abandonment, retry factor, and access-skew signals
- summarizes whether the system is merely slow or becoming unfair

Slot Inventory Sentinel Agent

- reads slot hold utilization and expiration pressure
- detects when access is clogged even if the core service is still technically up

Browser Sentinel Agent

- opens the Guardrail Console
- captures the visible incident state
- verifies that the browser-facing operator surface matches the metrics story

The output of these four agents becomes one normalized incident state.

7.2 Case-Library Retrieval

Before deciding, the system should retrieve similar prior cases for reference.

The case library should store:

- incident features
- shortlisted actions

- chosen action
- predicted result
- actual result
- verdict

Retrieval signal can be simple in Week 2:

- nearest scenario type
- closest retry / abandonment / completion pattern
- dependency status category

The point is not a perfect retrieval system. The point is to show that each incident is informed by prior cases, not treated as a blank slate.

7.3 Shortlist Mechanism

We should keep the design:

1. **Action library**
   - maintain a broader healthcare-access remediation library
2. **Scenario shortlist**
   - select the 4-6 most relevant actions for the current incident
3. **Guardrail ranking**
   - ask Stratus to rank only the shortlist

Shortlist mechanism for Week 2:

- use a **hybrid algorithmic shortlist builder**
- not a separate planning LLM agent

Inputs:

- scenario classification
- dependency status
- retry intensity
- slot-hold pressure
- abandonment severity
- regional saturation

Rules:

- always include the dangerous reflex if it is realistically tempting
- always include 1-2 conservative stabilizers
- include aggressive or disruptive actions only when severity crosses a threshold
- keep shortlist size at 4-6 so Stratus reasons over a tight, meaningful set

Why this is the right Week 2 choice:

- easier to debug than a free-form agentic selector
- deterministic enough for demo stability
- still rich enough for Tony to improve

7.4 Guardrail Decision

Stratus should rank the shortlist and explain:

- latency direction
- error direction
- retry-storm risk
- blast radius
- recovery confidence
- why the dangerous reflex is unsafe right now

The main product moment is:

- the system does not merely pick an action
- it rejects a locally reasonable but globally dangerous action before execution

7.5 Execution

Execution remains OpenClaw-first:

- OpenClaw is armed once through watch mode
- the watcher latches a new firing incident from the webhook path
- run `run.py --phase demo`
- read the demo artifact and browser playbook
- open `/openclaw-execution`
- inspect the before-action incident card
- click the single stable execution button
- run verify
- open the verdict surface

Browser fallback remains:

- if browser control fails, do not replan
- run `run.py --phase fallback`
- finish the report from the saved plan

Operational fallback inside the scenario remains:

- last-resort action can disable online scheduling and route to a manual call-center or callback workflow

7.6 Evaluation and Case Writeback

After action, the Evaluation Agent should measure:

- before vs after scheduling latency
- retry reduction
- abandonment change
- booking completion change
- blast radius
- predicted-vs-actual drift

Then it should write back:

- situation
- action
- result
- verdict

This is more useful than a binary success/failure log.

8. Action Library and Mappings

We should keep the existing action IDs for code stability, but present them with healthcare labels.

Core action mappings:

- `rate_limit_retries` -> **Throttle Booking Retries**
- `enable_payment_circuit_breaker` -> **Enable Eligibility Circuit Breaker**
- `increase_retry_backoff` -> **Increase Booking Retry Backoff**
- `restart_payment` -> **Restart Eligibility Service**
- `shift_traffic` -> **Shift Scheduling Traffic**
- `disable_flag` -> **Disable Online Scheduling**

Operational meaning:

- throttling retries reduces self-inflicted pressure
- circuit breaking fails fast on a degraded dependency
- retry backoff smooths load without a full shutdown
- restarting eligibility is a tempting but risky reflex
- traffic shift is only safe if spare capacity is real
- disabling online scheduling is a harsh but credible manual fallback

9. Frontend Product Surface

The frontend should now be explicitly split into two surfaces plus one execution page:

- **Telehealth Scheduling Stability Board** at `/operations`
- **Guardrail Console** at `/`
- **OpenClaw Execution Surface** at `/openclaw-execution`

Required product feel:

- the operations board should look like a real digital-access monitoring system
- the Guardrail Console should look like a real operator decision product
- the execution surface should look simple, stable, and browser-friendly for OpenClaw

Required views in the Guardrail Console:

- `Incident View`
- `Decision View`
- `Execution View`
- `Verdict View`

What must be true by the end of Week 2:

- all copy, labels, and business metrics speak healthcare-access language
- scenario switching works across the healthcare incident portfolio
- the operations board and control console feel visually distinct
- verdicts read like operational decisions, not debug logs

Judge-facing outcome:

- the system should look like a product an access operations team could plausibly use

10. Extensions

These are good extensions, but only after the one-action workflow is stable.

10.1 Optional Extension: Abnormality Localization

If time allows, add a second pass that pinpoints individual abnormalities such as:

- bot-like booking bursts
- slot-hold abuse or stuck holds
- regional skew
- specific dependency hotspots

Week 2 recommendation:

- implement this as a lightweight classifier or specialist agent output
- do not let it block the main one-action workflow

10.2 Optional Extension: Three-Action Sequence Planning

If time allows, add a sequence planner that proposes three actions in one go.

Default behavior:

- plan action 1, action 2, action 3
- execute only step 1 immediately
- continue with the sequence only if the post-action state supports it

Replanning rule:

- only ask Stratus to replan when the outcome is **partial** or **severe failure**

Week 2 priority:

- the one-action loop is primary
- three-action planning is explicitly secondary

11. Team Responsibilities

Everyone should touch both OpenClaw and Stratus, but primary ownership should stay clear.

Hansen

Primary ownership:

- Operator Agent
- execution flow
- Guardrail Console shell

Develop:

- own the OpenClaw execution handoff and browser playbook integration
- own the Execution View
- own overall console shell and navigation
- own scenario selection and sequence-progress presentation
- connect plan -> execute -> verify -> verdict into one product flow

Test:

- rehearse end-to-end browser runs
- test fallback-path reliability
- test console flow across all four views

Furnish:

- final demo script
- final OpenClaw prompt
- final shell polish
- scenario-switch UX and sequence-timeline UX

OpenClaw touchpoint:

- deepest owner of browser execution

Stratus touchpoint:

- render guardrail output clearly in Execution and Verdict

Frontend touchpoint:

- Guardrail Console shell + Execution View

End-of-week deliverable:

- a polished, demo-ready `Guardrail Console` and `OpenClaw Execution Surface` that can carry the final live run without explanation from another teammate

Tony

Primary ownership:

- Guardrail Agent
- shortlist mechanism
- Stratus ranking stability

Develop:

- own the `12–20 -> 4–6` shortlist mechanism
- stabilize Stratus output on shortlist ranking
- improve rationale quality around dangerous-reflex rejection
- tune how guardrail outputs are shaped for execution

Test:

- repeated Stratus runs on the healthcare incident portfolio
- compare shortlist rules and prompt variants
- maintain 实验记录 `TBD`

Furnish:

- final shortlist logic note
- final Stratus request/response schema
- concise explanation of why the reflex is unsafe

OpenClaw touchpoint:

- make guardrail outputs directly usable by OpenClaw execution

Stratus touchpoint:

- deepest owner of counterfactual ranking

Frontend touchpoint:

- Decision View ranking cards and shortlist explanation

End-of-week deliverable:

- a stable shortlist + Stratus decision layer whose outputs are clear enough to be shown directly on the `Decision View`

Charlie

Primary ownership:

- healthcare scenario truth
- incident portfolio
- product narrative

Develop:

- define the healthcare-access scenario portfolio
- own incident realism and case-library taxonomy
- define access/fairness semantics and visual acceptance criteria
- define the optional abnormality-localization taxonomy

Test:

- validate that scenarios feel real and distinct
- validate that chosen actions make sense per scenario
- validate that the narrative is clear to judges without extra explanation

Furnish:

- final Scenario A spec
- incident portfolio sheet
- evaluation rubric
- short positioning copy for the pitch

OpenClaw touchpoint:

- define what the Browser Sentinel and Verifier should look for

Stratus touchpoint:

- define the truth conditions Stratus should reason over

Frontend touchpoint:

- Incident View copy and scenario framing

End-of-week deliverable:

- a believable healthcare incident portfolio and Incident View narrative that make the product feel grounded from the first screen

Eason

Primary ownership:

- Evaluation Agent
- baseline mode
- case writeback

Develop:

- own baseline-mode comparison against the same shortlist
- own post-action evaluation and drift scoring
- own case-library writeback structure
- define how simulated healthcare metrics move after each action

Important independence rule:

- this work should remain buildable without waiting on Hansen integration changes

Test:

- show baseline chooses the obvious but worse reflex in Scenario A
- show guardrail path outperforms baseline on the same incident
- ensure comparison artifacts are readable in one screen

Furnish:

- final baseline artifact
- evaluation artifact
- case-library record format

OpenClaw touchpoint:

- surface evaluation outputs in the final OpenClaw-driven story

Stratus touchpoint:

- compare baseline vs guardrail on the same scenario and shortlist

Frontend touchpoint:

- Verdict View comparison and writeback messaging

End-of-week deliverable:

- a verdict/comparison layer that clearly shows why the guardrail beat the baseline and what was written back into the case library

12. Demo Prep

To stand out in the hackathon, the demo should emphasize:

1. **A dangerous reflex was rejected**
   - not just "an action was recommended"
2. **Patient access and fairness were protected**
   - not just uptime
3. **OpenClaw visibly acted**
   - the browser is part of the story
4. **Predicted vs actual was verified**
   - this is not fire-and-forget automation
5. **The agent woke up on its own**
   - it was armed in the background and activated from a real incident signal

Recommended demo sequence:

1. Trigger one healthcare-access incident live
2. Show that OpenClaw is already armed and waiting
3. Let the incident latch from the webhook path
4. Show the incident in the Guardrail Console
5. Turn on baseline mode and show it picks `restart_payment` shown as `Restart Eligibility Service`
6. Show Stratus ranking a safer action from the shortlist
7. Let OpenClaw execute the safer action
8. Verify the after-state
9. End on a verdict that explicitly says:
   - dangerous reflex rejected
   - safer action chosen
   - blast radius stayed low
   - case written back for future incidents

13. Grounding

This framing should be grounded in real scheduling and healthcare-IT concepts.

- HL7 FHIR `Appointment` and `Slot` resources justify the scheduling / slot-hold semantics
- Telehealth patient-preparation guidance supports realistic scheduling and manual-support flows
- Health IT contingency-planning guidance supports orderly fallback and restart discipline
- Open-source scheduling systems such as OpenEMR can inform future mirrors if we want a more realistic front-end shell later

Suggested source links:

- https://hl7.org/fhir/r4/appointment.html
- https://fhir.hl7.org/fhir/slot.html
- https://telehealth.hhs.gov/providers/preparing-patients-for-telehealth/helping-patients-prepare-for-their-appointment
- https://www.healthit.gov/sites/default/files/playbook/pdf/2-contingency-planning-final.pdf
- https://www.open-emr.org/wiki/index.php/OpenEMR_Wiki_Home_Page